Previo 9

SSH: Secure Shell

Cuestionario Previo

Universidad Nacional Autónoma de México.
Facultad de Ingeniería.

Laboratorio de Redes de datos seguras.

Profesora: *MTRA. MARÍA EUGENIA BAUTISTA GONZÁLEZ.*

Grupo: 08

Alumno:
320060829

Fecha de Entrega:

16 de Abril de 2026

Contents

1 ¿Qué es el reenvío por X11? 2

2 ¿Qué es un sniffer? 3

3 Mencione cuáles son las versiones del protocolo SSH y explique sus características. 3

4 ¿Cuáles son las secuencias de eventos a llevar a cabo en una conexión SSH? 4

5 ¿A qué nos referimos con la Autenticación? 5

6 Explique detalladamente los pasos que se producen cuando un cliente contacta a un servidor a través del protocolo SSH. 6

7 ¿Qué algoritmo de cifrado emplea el protocolo SSH? 7

8 ¿En dónde es conveniente utilizar SSH? 8

9 ¿Cuáles son los objetivos principales de la capa 6 del modelo OSI? 8

1 ¿Qué es el reenvío por X11?

El reenvío por X11 (X11 Forwarding) es un mecanismo avanzado de tunelización y encapsulación de red que permite a los usuarios ejecutar aplicaciones con interfaces gráficas de usuario (GUI) en un servidor remoto de alto rendimiento, mientras la visualización gráfica, los eventos de entrada del teclado y los movimientos del dispositivo apuntador se procesan y renderizan de manera interactiva en la pantalla de la máquina local del usuario [7]. Para comprender a cabalidad la importancia y el funcionamiento de este mecanismo, es imperativo analizar la arquitectura cliente-servidor única que define al Sistema de Ventanas X (X Window System), el cual es el estándar fundacional para las interfaces gráficas en sistemas operativos de la familia UNIX y distribuciones de Linux.

A diferencia de las arquitecturas gráficas monolíticas, el protocolo X11 fue diseñado desde sus inicios en los años ochenta con la transparencia de red como su objetivo principal. Esta filosofía introduce una terminología que frecuentemente resulta contraintuitiva: en el ecosistema X11, el "servidor X" (X Server) es el software que se ejecuta en la computadora local del usuario, controlando directamente el hardware de visualización (monitores) y los periféricos de entrada (teclado y ratón). Por otro lado, los "clientes X" (X Clients) son las aplicaciones individuales (por ejemplo, un navegador web o un software de modelado) que se ejecutan, computan y consumen recursos de procesamiento, las cuales pueden residir tanto en la máquina local como en un nodo remoto de computación [7].

En su configuración nativa, cuando un cliente X remoto necesita actualizar su interfaz gráfica, transmite comandos de dibujo y mapas de bits a través de la red hacia el servidor X local utilizando el protocolo de red X11, el cual típicamente escucha en los puertos TCP a partir del 6000. Esta transmisión directa presenta una vulnerabilidad de seguridad de proporciones críticas, ya que transmite la totalidad de sus datos en texto claro, sin ningún tipo de cifrado. Cualquier actor malicioso que intercepte el tráfico puede visualizar el contenido de las ventanas, capturar pulsaciones de teclas (keylogging) o inyectar eventos de interfaz falsificados.

El reenvío de X11 a través del protocolo Secure Shell (SSH) surge como la solución criptográfica a esta vulnerabilidad. Al invocar una conexión SSH con las directivas de configuración adecuadas (utilizando el comando `ssh -X` o `ssh -Y`), el demonio de SSH en el servidor remoto (`sshd`) intercepta la comunicación gráfica en su origen. En lugar de establecer una conexión insegura a Internet, el demonio remoto crea dinámicamente un canal de interconexión local (socket de dominio UNIX o un puerto de bucle cerrado) que actúa como una pantalla proxy ficticia [7].

Este comportamiento está íntimamente ligado a la manipulación de variables de entorno. El demonio SSH configura automáticamente la variable `$DISPLAY` en la sesión del usuario remoto, asignándole un valor artificial que apunta a la pantalla proxy local del servidor, como `localhost:10.0` o `:11`. Los usuarios no deben alterar manualmente la variable `$DISPLAY`, ya que esto destruiría la lógica de enrutamiento del túnel seguro. Una vez que la aplicación cliente X inicia, dirige todo su flujo de datos hacia el puerto proxy local. El demonio `sshd` captura este tráfico, lo somete a los rigurosos algoritmos de cifrado simétrico negociados y lo inyecta a través del canal multiplexado de la conexión SSH, garantizando su seguridad integral.

2 ¿Qué es un sniffer?

Un sniffer (analizador de protocolos de red o capturador de paquetes) es una herramienta especializada de software o un dispositivo físico de hardware diseñado específicamente para interceptar, registrar, decodificar y analizar el tráfico de datos binarios que circula a través de las interfaces de una red de telecomunicaciones [13]. Operando primordialmente en las capas inferiores del modelo OSI (Capa Física y Capa de Enlace de Datos), un sniffer funciona bajo el principio de la escucha pasiva, lo que significa que su objetivo intrínseco es monitorizar las interacciones electrónicas de manera completamente sigilosa, sin alterar la carga útil de los paquetes ni alertar a los dispositivos emisores o receptores sobre su presencia observacional [13].
El mecanismo técnico fundamental que habilita la funcionalidad de un analizador de protocolos es la manipulación de los controladores de bajo nivel de la Tarjeta de Interfaz de Red (NIC). Para que un sniffer pueda desempeñar su función, reconfigura el comportamiento del hardware de la tarjeta de red induciéndola a un estado operativo conocido como "modo promiscuo" en redes cableadas Ethernet, o "modo monitor" en redes inalámbricas Wi-Fi [13]. Al habilitar este modo, se suprime por completo el filtro de direcciones MAC, obligando al hardware a capturar y transferir a la CPU absolutamente todas las tramas detectadas en el medio físico, sin importar a qué nodo estén dirigidas originalmente.
La eficacia de la interceptación lograda por un sniffer está condicionada por la topología de la red:

- **Redes compartidas:** En infraestructuras antiguas (hubs) o redes Wi-Fi abiertas, todas las señales se replican físicamente a todos los nodos. En estos escenarios, un sniffer en modo promiscuo obtiene visibilidad total e instantánea del tráfico de la subred [13].

- **Redes conmutadas (Switches):** Un switch moderno aísla los dominios de colisión y reenvía las tramas de manera exclusiva al puerto de la máquina destino. Aquí, un sniffer conectado aleatoriamente solo captura su propio tráfico y el de multidifusión.

Para superar las restricciones en redes conmutadas y capturar tráfico de terceros, los atacantes recurren a tácticas como el Envenenamiento de Caché ARP (ARP Spoofing). Al falsificar identidades ARP, el atacante desvía el flujo de datos hacia su máquina (Ataque Man-in-the-Middle), permitiéndole al sniffer registrar y extraer información confidencial [13]. El valor de los sniffers es dual: de forma legítima, los administradores los usan para diagnosticar fallas y auditar la red; de forma maliciosa, los cibercriminales los emplean para extraer credenciales en texto plano, haciendo mandatorio el uso de cifrado fuerte como SSH o VPNs para proteger las comunicaciones.

3 Mencione cuáles son las versiones del protocolo SSH y explique sus características.

El protocolo Secure Shell (SSH) ha atravesado un riguroso proceso de evolución arquitectónica y estandarización criptográfica. Diseñado originalmente como una respuesta ante ataques de interceptación de contraseñas (sniffing) para reemplazar protocolos en texto claro como Telnet y FTP, SSH se clasifica en dos iteraciones principales y diametralmente distintas: SSH-1 y SSH-2 [1].

SSH-1: El Diseño Monolítico Original

La primera versión, desarrollada en 1995, operaba bajo un paradigma de diseño monolítico. Esto significa que todas las funciones (negociación de cifrado, autenticación del usuario, sesión y transmisión de datos) estaban fuertemente acopladas en un único flujo de protocolo indivisible. Sus características principales eran:

- Dependía del algoritmo asimétrico RSA para la autenticación de los hosts y la transmisión de claves de sesión.
- Para el cifrado masivo empleaba algoritmos simétricos como DES, 3DES e IDEA.
- Su principal debilidad radicaba en la comprobación de la integridad: utilizaba un control de redundancia cíclica (CRC-32), el cual no poseía propiedades criptográficas robustas, permitiendo a los atacantes modificar paquetes inyectando comandos sin invalidar la suma de comprobación.

SSH-2: Estandarización Modular por la IETF

Para mitigar las deficiencias, la IETF reescribió el protocolo, culminando en 2006 con la especificación de SSH-2 [2]. SSH-2 no es retrocompatible con SSH-1 y se caracteriza por su segmentación estricta en tres protocolos modulares, jerárquicos y funcionalmente aislados:

- **Protocolo de la Capa de Transporte (RFC 4253):** Se ejecuta sobre TCP/IP y es responsable de la negociación inicial de algoritmos, el intercambio de claves efímeras (Diffie-Hellman), la autenticación del servidor para evitar ataques MitM y el cifrado simétrico del flujo [4].
- **Protocolo de Autenticación de Usuario (RFC 4252):** Opera exclusivamente sobre la capa de transporte ya cifrada. Su función única es verificar matemáticamente la identidad del cliente, soportando contraseñas, claves públicas y métodos basados en el host de manera combinada [3].
- **Protocolo de Conexión (RFC 4254):** Administra la multiplexación del túnel cifrado, permitiendo que múltiples flujos lógicos independientes (como terminales PTY, reenvío X11 o SFTP) compartan la misma conexión TCP cifrada subyacente [5].

Adicionalmente, SSH-2 reemplazó el mecanismo CRC-32 por potentes Códigos de Autenticación de Mensajes (HMAC-SHA2), garantizando la integridad infalible de los datos [4].

4 ¿Cuáles son las secuencias de eventos a llevar a cabo en una conexión SSH?

La solidez operativa del protocolo Secure Shell radica en la rigurosa coreografía de su secuencia de inicio o "handshake" de la capa de transporte. Este proceso, que transita desde la capa física hasta la apertura de canales de aplicación, se ejecuta mediante una serie de pasos deterministas para evitar la intercepción [6].

1. **Establecimiento TCP y Verificación de Versiones:** El proceso se desencadena con un saludo de tres vías (3-way handshake) estándar de TCP al puerto 22. Una vez abierto, cliente y servidor intercambian cadenas de identificación en formato ASCII declarando explícitamente la versión del protocolo soportada, lo cual previene incompatibilidades de red [8].

2. **Negociación de Algoritmos (Algorithm Negotiation):** Ambas partes se transmiten listas ordenadas detallando los algoritmos matemáticos que soportan para el intercambio de claves, la autenticación, el cifrado simétrico masivo y la función hash (HMAC) para la integridad. Se acuerda el primer método mutuamente soportado en cada categoría [4].

3. **Ejecución del Intercambio de Claves (KEX):** Mediante algoritmos como Diffie-Hellman, cliente y servidor generan colaborativamente un secreto matemático compartido a través de la red vulnerable, sin que los componentes de este secreto se transmitan directamente. Derivan un secreto compartido (K) y un hash del intercambio (H) [8].

4. **Autenticación del Servidor mediante Clave de Host:** El servidor envía su clave pública permanente para identificarse y demuestra poseer la clave privada correspondiente firmando criptográficamente el hash del intercambio. El cliente verifica esta firma contra su archivo local de hosts conocidos (`~/.ssh/known_hosts`). Si hay discrepancia, se aborta la conexión [6].

5. **Transición al Túnel Cifrado (`NEWKEYS`):** Con el secreto validado, se derivan las claves simétricas finales y se emite el mensaje `SSH_MSG_NEWKEYS`. A partir de este momento, todo flujo posterior se realiza bajo el cifrado simétrico acordado, estableciendo el túnel de transporte inexpugnable [4].

5 ¿A qué nos referimos con la Autenticación?

En el dominio de la ciberseguridad, la autenticación es definida axiomáticamente como el proceso metódico, verificable y criptográficamente sólido de confirmación de que una entidad (ya sea un usuario humano o un proceso de software) es genuinamente quien o lo que afirma ser [12]. Es el mecanismo de control de acceso primario que establece la procedencia y la legitimidad de una identidad reclamada en un entorno digital. Se diferencia de la simple identificación (afirmar quién eres) y de la autorización (los permisos que tienes una vez validado).
Para garantizar la fiabilidad del proceso probatorio, la ingeniería de seguridad fundamenta la autenticación en tres categorías paradigmáticas de credenciales o "factores":

- **Factores de Conocimiento (Algo que el usuario sabe):** Se basan en un secreto conceptual compartido entre el usuario y el sistema, como las contraseñas alfanuméricas, frases de paso (passphrases) o pines numéricos.

- **Factores de Posesión (Algo que el usuario tiene):** Requieren la presentación de un artefacto físico o un token digital vinculando la identidad a un objeto material. Ejemplos incluyen aplicaciones de Contraseñas de Un Solo Uso (OTP), tokens de hardware o llaves de seguridad físicas (FIDO).

- **Factores de Inherencia (Algo que el usuario es):** Constituyen el nivel de mayor especificidad, fundamentándose en los atributos biológicos o conductuales del individuo, tales como huellas dactilares, reconocimiento del iris o topología facial.

En el contexto estricto de SSH, la autenticación opera bajo el paradigma de la **Autenticación Mutua** o bilateral. Antes de que el usuario envíe sus credenciales encriptadas, el servidor debe autenticarse primero ante el cliente demostrando la posesión de una clave privada de host válida. Solo entonces el cliente envía sus pruebas de identidad, asegurando que la confianza es recíprocamente confirmada en cada transacción [12].

6 Explique detalladamente los pasos que se producen cuando un cliente contacta a un servidor a través del protocolo SSH.

Habiendo superado la rigurosa fase de inicialización de la Capa de Transporte, donde se negociaron los algoritmos y se estableció el túnel cifrado, la conexión SSH transita hacia sus fases operativas para validar al usuario y abrir sesiones. Esta secuencia se divide en dos fases modulares detalladas en los RFC 4252 y 4254 [3, 5].

Fase 1: El Protocolo de Autenticación de Usuario (`ssh-userauth`)

El objetivo de esta fase es verificar de manera concluyente los derechos de acceso de la persona o proceso que controla al cliente SSH [3]:

1. **Solicitud Inicial y Negociación:** El cliente emite una petición encriptada declarando el nombre de usuario. El servidor responde con un mensaje `SSH_MSG_USERAUTH_FAILURE` que incluye una lista de los métodos que está dispuesto a aceptar para ese usuario (ej. `publickey`, `password`, `keyboard-interactive`).

2. **Verificación de Identidad por el Cliente:** El cliente elige un método soportado:

   - *Mecanismo de Contraseña:* Si se elige, el cliente solicita la contraseña al usuario en la terminal y la inyecta cifrada dentro del túnel de transporte, evadiendo ataques de red.
   - *Mecanismo de Clave Pública:* Es el método preferencial y robusto. El cliente ensambla un mensaje con el identificador de la sesión, lo firma matemáticamente con su clave privada y lo envía al servidor. El servidor verifica la firma contra el archivo `~/.ssh/authorized_keys`. Si es válida, el acceso se concede sin transmitir secretos [6].

3. **Concesión:** Si la autenticación es exitosa, el servidor emite el mensaje `SSH_MSG_USERAUTH_SUCCESS` transfiriendo el control a la fase de conexión.

Fase 2: El Protocolo de Conexión y Multiplexación (`ssh-connection`)

Culminada la autenticación, la arquitectura SSH utiliza la multiplexación para gestionar flujos lógicos independientes sobre el mismo túnel [5]:

1. **Apertura de Canales Lógicos:** El cliente solicita la apertura de canales virtuales mediante `SSH_MSG_CHANNEL_OPEN`. Puede crear simultáneamente un pseudo-terminal interactivo (PTY) para línea de comandos, canales para el reenvío de puertos X11 o sesiones de SFTP.

2. **Control de Flujo:** Cada canal maneja sus propias cuotas de memoria interna y mecanismos de control de flujo. Los datos no se envían precipitadamente hasta que el nodo opuesto notifica que tiene capacidad para procesarlos.

3. **Interacción y Cierre:** Los usuarios interactúan fluidamente con la consola remota protegida. Al finalizar, el cliente envía solicitudes de cierre para demoler ordenadamente cada canal (`SSH_MSG_CHANNEL_CLOSE`) y, finalmente, desmantelar el túnel TCP [5].

7 ¿Qué algoritmo de cifrado emplea el protocolo SSH?

El protocolo Secure Shell no emplea un único algoritmo, sino una suite criptográfica (cipher suite) que despliega de forma orquestada algoritmos asimétricos, simétricos y de hash en diferentes momentos de la conexión [11].

- **Cifrado Asimétrico (KEX y Autenticación):** Utilizado en la etapa fundacional para negociar secretos matemáticos y validar la identidad.
  - Para el *Intercambio de Claves*, se basa en Diffie-Hellman (DH) o su variante de Curva Elíptica (ECDH), como `curve25519-sha256`, proveyendo secreto perfecto hacia adelante.
  - Para las *Firmas de Host*, SSH históricamente utilizó RSA, pero debido al alto costo computacional de sus claves, está migrando masivamente hacia el Algoritmo de Firma Digital (DSA) y a la Criptografía de Curva Elíptica (ECC), como `Ed25519`. ECC ofrece seguridad formidable utilizando claves reducidas, optimizando el rendimiento frente a la futura computación cuántica [11].

- **Cifrado Simétrico (Flujo de Datos Masivo):** Una vez derivado el secreto compartido, el túnel utiliza algoritmos simétricos rápidos para cifrar el contenido interactivo. El estándar dominante es el *Advanced Encryption Standard (AES)* en modos CTR o GCM (con claves de 128 a 256 bits). Alternativamente, SSH soporta algoritmos de flujo como ChaCha20 o arquitecturas ultra seguras de bloques como *Twofish* [11].

- **Códigos de Autenticación de Mensajes (HMAC):** Para asegurar la integridad de cada fragmento frente a inyecciones de red, se emplean algoritmos hash de la familia SHA-2 (HMAC-SHA2-256 o HMAC-SHA2-512) anexando una firma criptográfica efímera a cada paquete [4].

Transición Hacia la Criptografía Poscuántica (PQC)

Ante la amenaza inminente de los ordenadores cuánticos (capaces de romper algoritmos como RSA y ECC mediante el algoritmo de Shor), el NIST ha estandarizado nuevos

algoritmos de encriptación poscuántica fundamentados en retículos matemáticos [9]. Estas normas (como FIPS 203 ML-KEM / Kyber para encapsulación de claves y FIPS 204 ML-DSA / Dilithium para firmas digitales) comenzarán a ser integradas en las infraestructuras de SSH para sustituir la criptografía asimétrica actual y resistir ataques del tipo "cosechar ahora, descifrar después" [10].

8 ¿En dónde es conveniente utilizar SSH?

El protocolo SSH es una herramienta indispensable y versátil en el campo de las telecomunicaciones y el despliegue de infraestructuras IT. Es operativamente conveniente, y frecuentemente mandatorio, utilizarlo en los siguientes escenarios [1]:

1. **Administración Centralizada y Terminales Remotos:** Su uso principal es proveer una vía cifrada y resistente a interceptaciones para que los administradores accedan a consolas de línea de comandos (CLI) de servidores Linux/Unix, routers o instancias en la nube pública desde cualquier lugar geográfico.

2. **Transferencia de Archivos Transaccional:** Al sustituir protocolos obsoletos como FTP o TFTP, SSH sustenta herramientas integradas como SFTP y SCP. Estos comandos aprovechan la infraestructura criptográfica de SSH para asegurar que código fuente, respaldos de bases de datos o credenciales no queden expuestos durante el envío [1].

3. **Reenvío de Puertos y Tunelización:** SSH puede encapsular tráfico de aplicaciones legadas inseguras. Permite el Reenvío Local (para acceder a puertos internos como bases de datos ocultas), el Reenvío Remoto (para exponer aplicaciones de desarrollo locales a Internet) y Proxies Dinámicos SOCKS, logrando conectar dispositivos en redes corporativas con firewalls sin modificar configuraciones críticas a nivel de routers.

4. **Automatización y Entornos CI/CD:** SSH permite el acceso distribuido mediante llaves asimétricas (sin contraseña), facilitando que plataformas industriales de orquestación (como Ansible) o pipelines de integración continua configuren centenares de servidores o auditen el rendimiento de manera automatizada y en tiempo real.

5. **Despliegue de Pasarelas de Salto (Bastion Hosts):** En redes segmentadas (como en AWS o centros de datos de alta seguridad), SSH se utiliza para configurar un servidor perimetral único y endurecido (Jump Server). Los administradores primero inician sesión en la pasarela SSH, y desde ahí acceden a los recursos de bases de datos privados, mitigando enormemente la superficie de exposición pública.

9 ¿Cuáles son los objetivos principales de la capa 6 del modelo OSI?

La Capa 6 o Capa de Presentación del Modelo OSI interactúa como el nexo inmediato entre la Capa de Sesión y la Capa de Aplicación. Su objetivo filosófico fundamental es aislar a las aplicaciones de red de las arduas tareas asociadas a las discrepancias

en los formatos de codificación, operando como un traductor universal que asegura la interoperabilidad entre dispositivos de fabricantes heterogéneos [15].
Los objetivos estructurales de la Capa de Presentación se ejecutan según la dirección del tráfico y se dividen en tres pilares principales:

1. **Formateo, Traducción de Datos y Representación:** Dado que los sistemas operativos difieren en la interpretación binaria de los datos (sistemas mainframe usando EBCDIC vs. computadoras modernas usando ASCII o Unicode), la Capa 6 asume la labor de abstraer la información. Serializa los datos desde la aplicación emisora a un formato común de transferencia estandarizado de red (como ASN.1) y, al llegar al destino, revierte el proceso, ajustando los datos a la sintaxis local esperada por el software consumidor [14].

2. **Compresión Lógica de la Información:** Con el mandato de optimizar los recursos limitados del medio físico, esta capa aplica rutinas intensivas de compresión matemática de datos (como multimedia, texto repetitivo o bases de datos) antes de descender a las capas inferiores. Esto maximiza el ancho de banda efectivo y reduce la latencia en las transferencias intercontinentales [15].

3. **Cifrado y Descifrado Estructural:** En la arquitectura original de OSI pura, la Capa 6 era la frontera lógica destinada a ejecutar los mecanismos operativos de la criptografía. Intercepta la información en texto claro producida por la Capa 7, aplica primitivos matemáticos (cifrado asimétrico/simétrico) y entrega la carga ininteligible a la capa de sesión, protegiendo así la privacidad frente a interceptaciones durante su trayecto en red [15].

References

[1] Stackscale. (2021). *Protocolo SSH: uso, versiones e implementaciones populares*. Recuperado de https://www.stackscale.com/es/blog/protocolo-ssh/

[2] Ylonen, T., & Lonvick, C. (2006). *RFC 4251: The Secure Shell (SSH) Protocol Architecture*. Internet Engineering Task Force (IETF). Recuperado de https://datatracker.ietf.org/doc/html/rfc4251

[3] Ylonen, T., & Lonvick, C. (2006). *RFC 4252: The Secure Shell (SSH) Authentication Protocol*. Internet Engineering Task Force (IETF). Recuperado de https://datatracker.ietf.org/doc/html/rfc4252

[4] Ylonen, T., & Lonvick, C. (2006). *RFC 4253: The Secure Shell (SSH) Transport Layer Protocol*. Internet Engineering Task Force (IETF). Recuperado de https://datatracker.ietf.org/doc/html/rfc4253

[5] Ylonen, T., & Lonvick, C. (2006). *RFC 4254: The Secure Shell (SSH) Connection Protocol*. Internet Engineering Task Force (IETF). Recuperado de https://datatracker.ietf.org/doc/html/rfc4254

[6] Red Hat Inc. (s.f.). *Secuencia de eventos de una conexión SSH - Manual de Referencia de Red Hat Linux 9*. Recuperado de https://legacy.redhat.com/pub/redhat/linux/9/en/doc/RH-DOCS/rhl-rg-es-9/s1-ssh-conn.html

[7] IBM Corporation. (s.f.). *Enviar un trabajo con reenvío SSH X11*. IBM Spectrum LSF Knowledge Center. Recuperado de https://www.ibm.com/docs/es/spectrum-lsf/10.1.0?topic=jobs-submit-job-ssh-x11-forwarding

[8] Cisco Systems. (2025). *Comprensión del intercambio de paquetes de Secure Shell (SSH)*. Recuperado de https://www.cisco.com/c/es_mx/support/docs/security-vpn/secure-shell-ssh/222335-understand-secure-shell-packet-exchange.html

[9] National Institute of Standards and Technology (NIST). (2024, 13 de agosto). *NIST Releases First 3 Finalized Post-Quantum Encryption Standards*. Recuperado de https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards

[10] Keyfactor. (s.f.). *Three new PQC algorithms are expected from NIST: Unpacking the news*. Recuperado de https://www.keyfactor.com/es/blog/unpacking-news-from-nist-three-new-algorithms-are-expected-in-2024/

[11] RedesZone. (s.f.). *Criptografía: Algoritmos de clave simétrica y asimétrica recomendados*. Recuperado de https://www.redeszone.net/tutoriales/seguridad/criptografia-algoritmos-clave-simetrica-asimetrica/

[12] G.A.G.C, Universidad Michoacana. (s.f.). *Redes de Computo - Definición de Autenticación y Ataques Informáticos*. Recuperado de https://www.fcca.umich.mx/descargas/apuntes/academia%20de%20informatica/Redes%20de%20Computo%20%20G.A.G.C/Unidad-3-4-5.pdf

[13] cdmon. (s.f.). *Sniffers: del uso legítimo a las amenazas ocultas en redes*. Recuperado de https://www.cdmon.com/es/blog/sniffers-del-uso-legitimo-a-las-amenazas-ocultas

[14] Amazon Web Services. (s.f.). *¿Qué es el modelo OSI? - Explicación de las 7 capas del modelo OSI*. Recuperado de https://aws.amazon.com/es/what-is/osi-model/

[15] Proofpoint. (s.f.). *Las 7 capas del modelo OSI y sus funciones*. Recuperado de https://www.proofpoint.com/es/threat-reference/osi-model